

FLStock — structure, slots and meaning

a4a Training Course — Module 1.3

Danai Mantopoulou Palouka & Alessandro Ligas

2026-09-13

What an FLStock is

An **FLStock** is a container holding your whole perception of the stock.

- **17 FLQuant slots** — all sharing the same ages and years
- plus name, desc and range

 Nothing new to learn

Every slot is an **FLQuant**, exactly as in Module 1.2. An **FLStock** is just a labelled box of them.

The 20 slots, grouped

Group	Slots
Catch	catch, catch.n, catch.wt
Landings	landings, landings.n, landings.wt
Discards	discards, discards.n, discards.wt
Biology	m, mat, m.spwn, harvest.spwn
Population	stock, stock.n, stock.wt
Exploitation	harvest
Metadata	name, desc, range

The pattern: total, numbers, weight

Catch, landings and discards each come as **three** slots:

Slot	Meaning	Dimensions	Units
catch	total weight landed + discarded	$1 \times \text{years}$	t
catch.n	numbers at age	$\text{ages} \times \text{years}$	1000
catch.wt	mean weight at age	$\text{ages} \times \text{years}$	kg

Note the first one is a **single row** (age = "all"), not a matrix.

Biology — what you must supply

Slot	Meaning	Typical
<code>m</code>	natural mortality at age	assumed, e.g. 0.1
<code>mat</code>	proportion mature at age	0 → 1 ogive
<code>m.spwn</code>	fraction of M before spawning	0
<code>harvest.spwn</code>	fraction of F before spawning	0

`m.spwn` and `harvest.spwn` say *how far into the year* spawning happens. Both 0 means spawning on 1 January.

Population and exploitation — what the model gives back

Slot	Meaning	Who fills it
stock.wt	mean weight at age in the stock	you
stock.n	numbers at age in the stock	the model
stock	total stock biomass	the model
harvest	fishing mortality at age	the model

The key distinction

You supply **14** of the 17 FLQuant slots. `stock`, `stock.n` and `harvest` are what `sca()` **estimates** — leave them empty.

range — seven numbers that matter

`range(stk)`

min	max	plusgroup	minyear	maxyear	minfbar	maxfbar
1	10	10	1998	2017	2	6

- min, max, plusgroup — the age structure
- minyear, maxyear — the time span
- minfbar, maxfbar — **which ages are averaged** to report $\bar{F}$

Metadata is not decoration

```
name(stk)
```

```
[1] "MUT"
```

```
desc(stk)
```

```
[1] "MUT in GSA32. WGSAD 2018"
```

Six months from now, this is how you know which run you are looking at.

Consistency: the model trusts you

Nothing checks that your slots agree with each other. You must ensure:

```
catch.n == landings.n + discards.n
```

```
catch == landings + discards
```

And `catch.wt` is the **catch-weighted** mean of the two:

```
catch.wt <- (discards.n / catch.n) * discards.wt +  
            (landings.n / catch.n) * landings.wt
```



Not a plain average

Averaging `landings.wt` and `discards.wt` directly is wrong — discards are small fish, and there are usually many more of them.

A trap worth knowing

FLQuant matches dimnames **by name**, not by position:

```
# year written first, but you still get ages x years  
FLQuant(x, dimnames = list(year = 1998:2017, age = 1:10))
```

That works. But the **values** fill in array order — quant first.



This only gives the right answer because the CSV is sorted **age within year**. A year-major file produces a silently **transposed** matrix, with no error. Always spot-check one value: `flq["3", "2005"]` against the raw file.

Checking your object

```
summary(stk)      # dimensions and ranges, slot by slot
plot(stk)         # the four-panel overview
units(stk)        # are the units what you think?
validObject(stk)  # does it satisfy the class definition?
verify(stk)       # FLCore's sanity checks
```

Summary

- An FLStock is **17 FLQuants** plus name, desc, range
- Totals are 1 x years; numbers and weights are ages x years
- You supply **14**; stock, stock.n, harvest come from the model
- range carries the **fbar ages** — a choice you must make
- Nothing enforces internal consistency: **that is your job**

Next: **Module 1.4** — FLIndex, the survey side of the data.

Exercise

Open `handson_M1.3_flstock.R` and build the MUT GSA32 stock from CSV files.

1. Read each file and turn it into an `FLQuant`
2. Derive `catch`, `catch.n` and `catch.wt` from landings and discards
3. Fill the biology slots
4. Assemble the `FLStock`, then set `range`, `name`, `desc` and `units`
5. Plot it

Why a class and not a list?

FLStock is an S4 class with a **validity function**. Every assignment through an accessor re-checks it:

- all FLQuant slots must share the same year and iter dimensions
- the quant dimension must match, except for the 1 x years totals

So `catch.n(stk) <- wrong_shape` fails **immediately**, at the line that caused it — rather than fifty lines later inside `sca()`, where the error message would tell you nothing useful.